

University of Asia Pacific
Department of Computer Science and Engineering
CSE 402: Operating System Lab

Lab Report: 08

Report Title: Page Replacement Algorithm

Submitted by:

Name **:** Md. Muhaimin Al Baki Raiyan

Student ID : 23101163

Section **:** C

Submitted to:

Atia Rahman Orthi

Lecturer,

Department of CSE, UAP

Date of Submission: 17/10/2026

1. Objective

• To understand why page replacement is needed when physical memory frames are limited.
• To implement the FIFO (First In First Out) page replacement algorithm in Python.
• To count page hits and page faults and calculate the hit ratio and fault ratio.

2. Theory

In paging, a process is divided into pages and main memory is divided into frames of the same size. Only some pages of a process can be in memory at a time. When the CPU refers to a page:

• **Page Hit:** the page is already in a frame, so it is used directly.
• **Page Fault:** the page is not in memory, so it must be loaded from secondary storage. If all frames are full, one existing page has to be removed. The rule used to pick that page is the page replacement algorithm.

2.1 FIFO Page Replacement

FIFO removes the page that has been in memory for the longest time, i.e. the page that was loaded first. It can be implemented with a queue, or (as in this lab) with a pointer that moves around the frames in a circular way. FIFO is simple, but it does not look at how often or how recently a page is used.

Hit Ratio = (Number of Hits / Total References) × 100%
Fault Ratio = (Number of Faults / Total References) × 100%

3. Problem Statement

Reference String	1, 2, 3, 4, 2, 3, 1
Number of Frames	3

Simulate FIFO page replacement for the data above. For each reference decide whether it is a hit or a fault, replace the oldest page when the frames are full, and finally find the total hits, total faults, hit ratio and fault ratio.

4. Algorithm

1. Start with empty memory, *first* = 0, hit = 0, fault = 0.
2. For each page *p* in the reference string:
   a. If *p* is in memory, increase hit.
   b. Otherwise increase fault. If a frame is free, put *p* in it; else put *p* at position *first* and set first = (first + 1) mod frames.
3. Calculate hit ratio and fault ratio and print the results.

5. Source Code

```
ref = [1, 2, 3, 4, 2, 3, 1]
n = 3

memory = []
first = 0
hit = 0
miss = 0

print("Page   Frames        Result")
for p in ref:
    if p in memory:
        hit += 1
        result = "Hit"
    else:
        miss += 1
        result = "Fault"
        if len(memory) < n:
            memory.append(p)
        else:
            memory[first] = p
            first = (first + 1) % n
    print(" ", p, "   ", str(memory).ljust(12), result)

total = len(ref)
print()
print("Total Hits   :", hit)
print("Total Faults :", miss)
print("Hit Ratio    :", round(hit / total * 100, 2), "%")
print("Fault Ratio  :", round(miss / total * 100, 2), "%")
```

6. Output

```
Run   23101163_fifo_page_replacement

C:\Users\raiyan\PycharmProjects\OSLab> python 23101163_fifo_page_replacement.py

Page   Frames        Result
  1     [1]         Fault
  2     [1, 2]      Fault
  3     [1, 2, 3]   Fault
  4     [4, 2, 3]   Fault
  2     [4, 2, 3]   Hit
  3     [4, 2, 3]   Hit
  1     [4, 1, 3]   Fault

Total Hits   : 2
Total Faults : 5
Hit Ratio    : 28.57 %
Fault Ratio  : 71.43 %

Process finished with exit code 0
```

Figure 1: Program output

7. Manual Simulation

The table shows the content of each frame after every reference. **F** = page fault, **H** = page hit.

Reference	1	2	3	4	2	3	1
Frame 1	1	1	1	4	4	4	4
Frame 2		2	2	2	2	2	1
Frame 3			3	3	3	3	3
Status	F	F	F	F	H	H	F

Table: Frame contents after each reference (shaded cell = newly replaced page)

8. Result

Total References	Page Hits	Page Faults	Hit Ratio	Fault Ratio
7	2	5	2/7 = 28.57%	5/7 = 71.43%

9. Discussion

Pages 1, 2 and 3 were loaded into empty frames, so the first three references were all faults (compulsory misses). When page 4 came, all frames were full and page 1 was replaced because it was loaded first. Pages 2 and 3 were then found in memory, giving the only two hits. At the last reference, page 1 was needed again; since it had already been removed it caused another fault and replaced page 2, which was now the oldest page.

In my program I did not shift the list every time a page is removed. Instead, the variable *first* keeps the position of the oldest page and moves forward circularly after each replacement. Because of this the order of pages inside the list is different from a queue (for example [4, 1, 3] at the end), but the set of pages in memory is exactly the same as normal FIFO.

The result also shows the main weakness of FIFO: page 1 was removed only because it was old, even though it was used again soon. An algorithm like LRU, which looks at recent use, would avoid some of these faults. FIFO can also suffer from Belady's anomaly, where giving more frames may increase the number of faults.

10. Conclusion

The FIFO page replacement algorithm was successfully implemented in Python for the reference string 1, 2, 3, 4, 2, 3, 1 with 3 frames. The simulation gave 2 page hits and 5 page faults, with a hit ratio of 28.57% and a fault ratio of 71.43%. FIFO is easy to implement, but since it ignores page usage it can replace pages that are still needed, so other algorithms like LRU or Optimal usually give better results.